

Μεταγλωττιστές 2025-26

Συντακτική ανάλυση και δημιουργία Abstract
Syntax Tree (AST)

(και παράδειγμα διερμηνείας)

Έξοδος συντακτικής (και σημασιολογικής) ανάλυσης

- «Ενδιάμεση αναπαράσταση» (intermediate representation, IR)
 - Το «**νόημα**» του προγράμματος
 - Σε συστηματική αναπαράσταση
 - Έτοιμο για επεξεργασία από τα επόμενα στάδια του μεταγλωττιστή
 - Στάδια βελτιστοποίησης
 - Παραγωγή εκτελέσιμου κώδικα
 - Διερμηνεία (απ' ευθείας εκτέλεση)

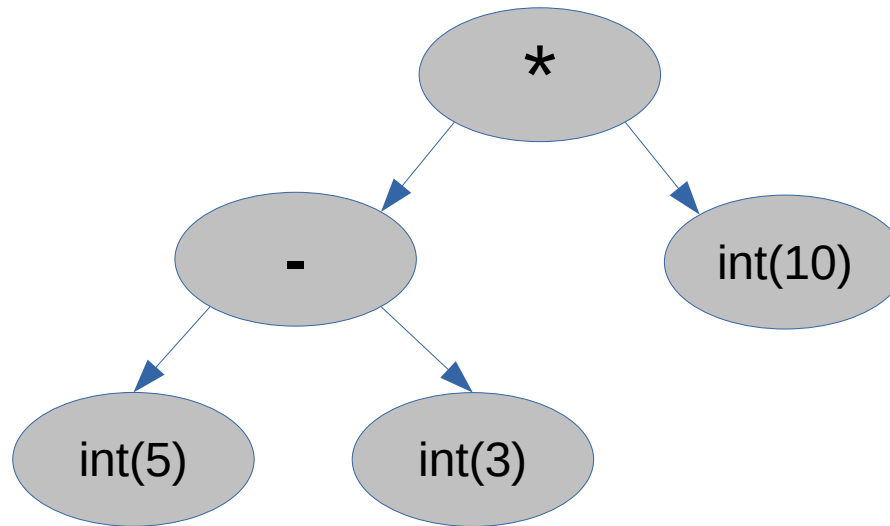
Abstract Syntax Tree (AST)

- Μια κλασική **δενδρική** μορφή «ενδιάμεσης αναπαράστασης» (IR)
 - Σε αντίθεση με ένα δένδρο παραγωγής (ανάλυση μη τερματικών συμβόλων γραμματικής μέχρι να καταλήξουμε στα φύλλα – tokens), **ένα AST περιέχει μόνο τη χρήσιμη πληροφορία**
 - Οργανωμένη με τέτοιον τρόπο ώστε να μπορεί να δοθεί ως είσοδος στα **επόμενα στάδια** του μεταγλωττιστή
- Δεν υπάρχει μία και μοναδική «στάνταρντ» μορφή AST, είναι περισσότερο μια «**εσωτερική δομή**» του κάθε μεταγλωττιστή
 - Υλοποιείται με τις διαθέσιμες δομές της γλώσσας που έχει γραφεί ο μεταγλωττιστής

Παράδειγμα ενδιάμεσης αναπαράστασης AST

- Είσοδος: $(5-3)*10$

Οι παρενθέσεις π.χ. βοηθούν στο να φτιάξουμε αυτό το συγκεκριμένο AST αλλά **δεν περιλαμβάνονται** στο AST



- Κάθε κόμβος εκφράζει μια αριθμητική τιμή
 - Αν είναι **φύλλο**, εκφράζει την τιμή μιας αριθμητικής σταθεράς (5, 3 και 10)
 - Αν είναι **ενδιάμεσος κόμβος**, εκφράζει την τιμή που υπολογίζεται από την εφαρμογή του αντίστοιχου τελεστή (- και *) στις τιμές των παιδιών του

Ο στόχος

- Δημιουργία **AST** από τη γραμματική των αριθμητικών εκφράσεων και εντολών ανάθεσης/εκτύπωσης
 - Μέρος του AST αναπαριστά την αριθμητική έκφραση
 - Το υπόλοιπο AST αναπαριστά την ακολουθία εντολών (ανάθεσης ή εκτύπωσης), με τη σειρά που δίνονται στον πηγαίο κώδικα

Η κλάση ASTNode

- Θα τη βρείτε στο module **compilerlabs**
- Βοηθητική κλάση για τη δημιουργία AST

```
from compilerlabs import ASTNode
```
- Παρέχει μεθόδους για την κατασκευή **κόμβων** ενός δένδρου AST (**AST nodes**)
 - Με τα επιλεγμένα χαρακτηριστικά (**attributes**)
 - Και προσδιορίζει τη σύνδεση κάθε κόμβου με τα επιθυμητά «παιδιά» του (**subnodes**)

Παράδειγμα

- Κατασκευή 2 «φύλλων» του AST (a και b)

```
a = ASTNode(attributes={'name': 'a'})
```

```
b = ASTNode(attributes={'name': 'b'})
```

- Κατασκευή κόμβου c και διασύνδεση των a και b ως «παιδιά» του

```
c = ASTNode(subnodes=[a,b], attributes={'name': 'c'})
```

- Εμφάνιση δένδρου που έχει το c ως ρίζα (για debugging)

```
print(c)
```

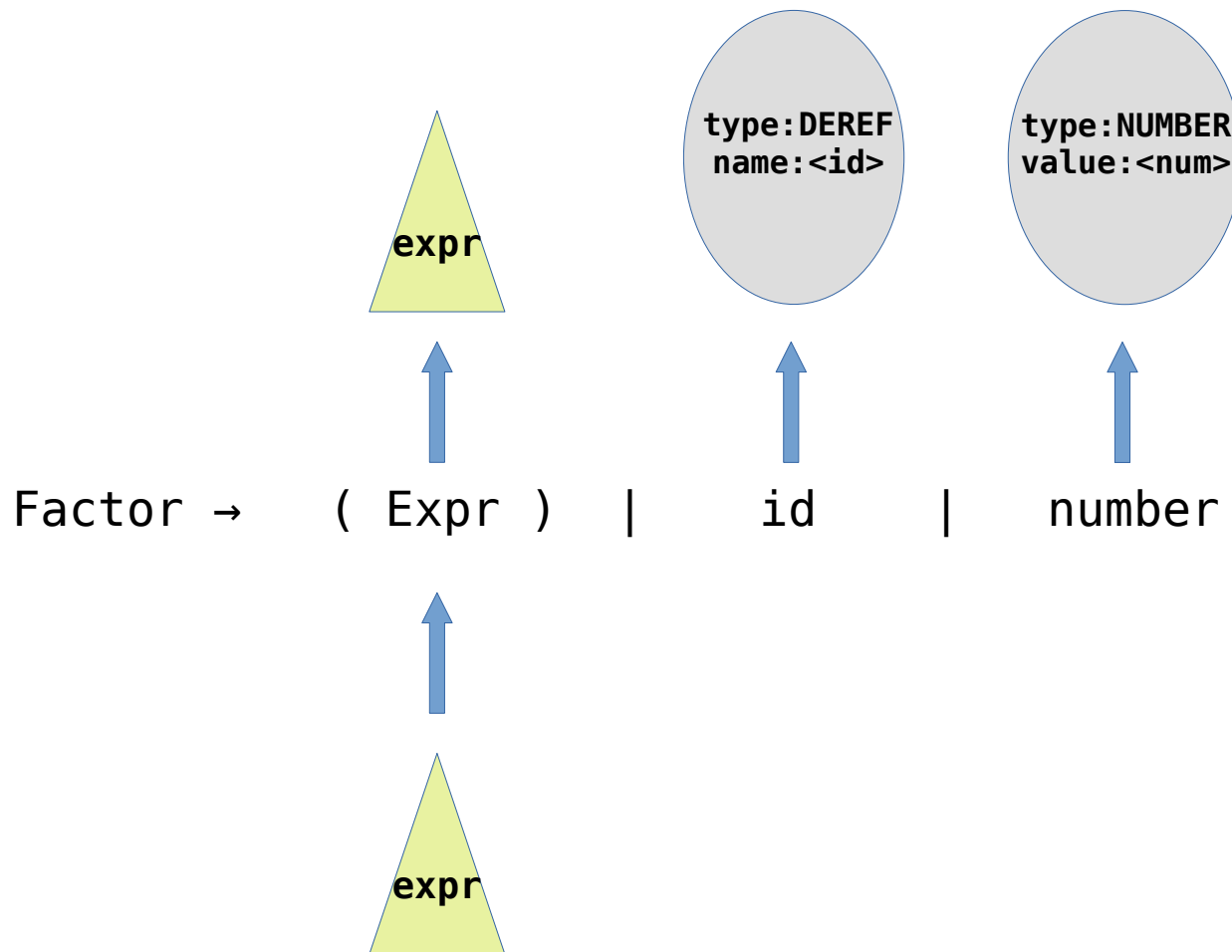
```
ASTNode{'name': 'c'}
```

```
├── ASTNode{'name': 'a'}
```

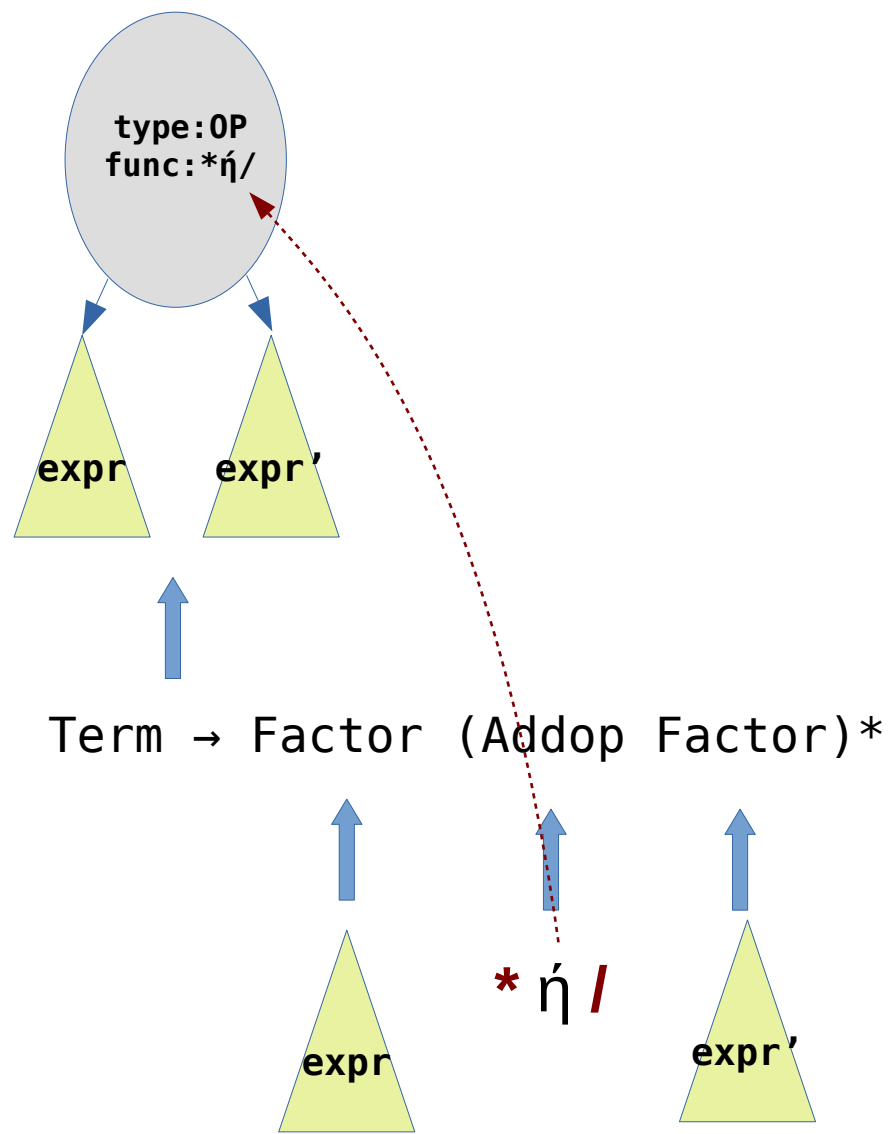
```
└── ASTNode{'name': 'b'}
```

AST αριθμητικής έκφρασης

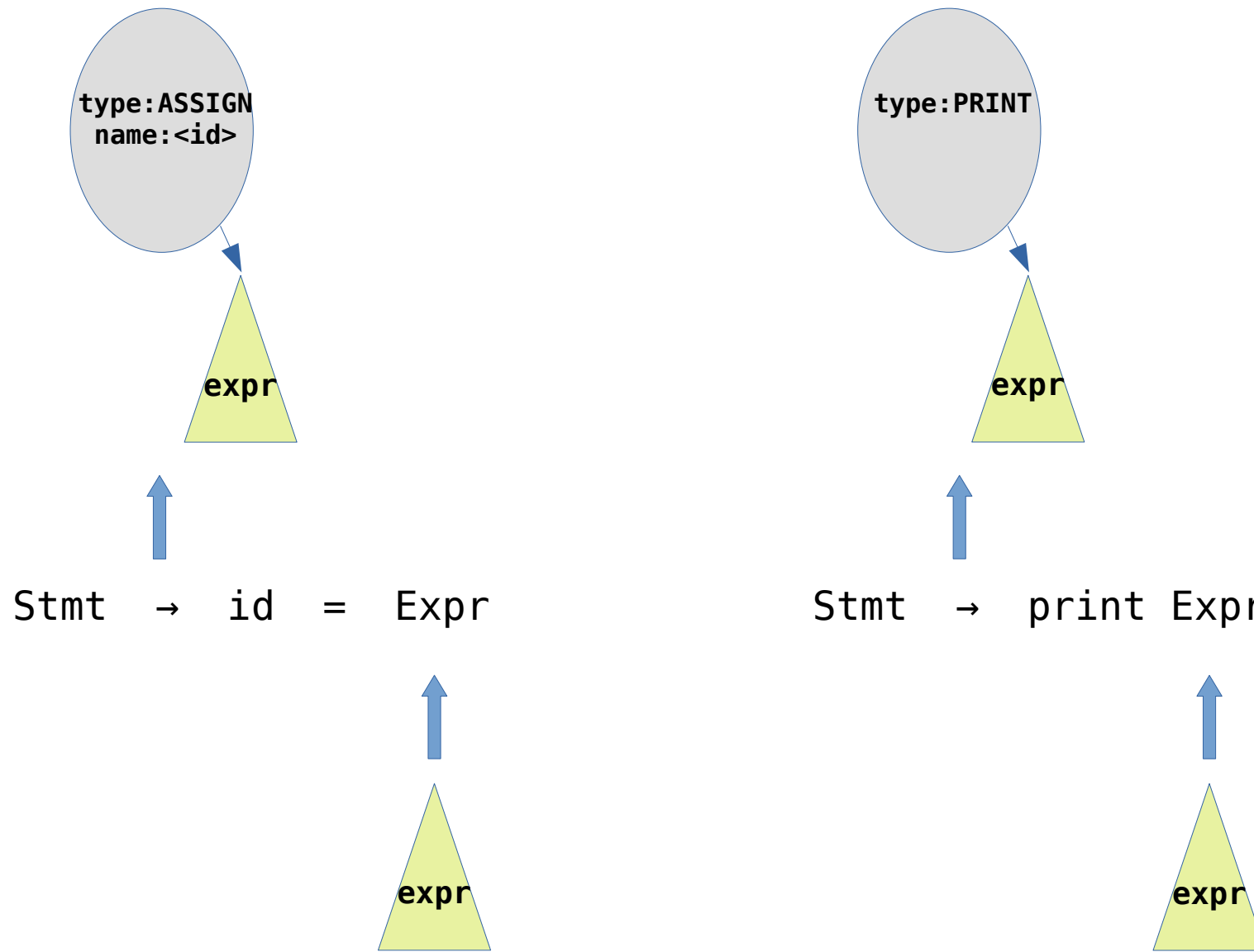
- Τα φύλλα θα δώσουν **float τιμές** (σταθερές και μεταβλητές)
- Οι ενδιάμεσοι κόμβοι είναι **αριθμητικοί τελεστές** που συνδυάζουν τις τιμές των **υποδένδρων** τους
- Κάθε μέθοδος που υλοποιεί κανόνες μη τερματικού συμβόλου της γραμματικής
 - Δέχεται μέρος του AST (**υποδένδρα**) από τις υπο-κλήσεις μεθόδων των μη τερματικών συμβόλων που περιέχει
 - Συνδυάζει τα υποδένδρα AST και επιστρέφει το **νέο AST** που προκύπτει
 - Η **Expr()** θα επιστρέψει το τελικό AST της κάθε αριθμητικής έκφρασης
 - Τα **Addop** και **Multop** συνεχίζουν να επιστρέφουν τον τελεστή



- Η μέθοδος του **Factor** κατασκευάζει τα **φύλλα** του AST
 - Μεταβλητές και σταθερούς αριθμούς float
 - Έχουν προωθεί ένα ήδη κατασκευασμένο υπο-AST από τη συνάρτηση του **Expr** στην περίπτωση του **(Expr)**



- Η **Term()** «χτίζει» επαναληπτικά (**while**) ένα AST όσο υπάρχουν τελεστές * ή /
- Το ίδιο συμβαίνει με την **Expr()** για τους τελεστές + ή -



- Η **Stmt** επιστρέφει κόμβους του AST που αντιστοιχούν στις **εντολές** (statements) της γλώσσας (**ανάθεση** ή **εκτύπωση**)

Stmt_list()

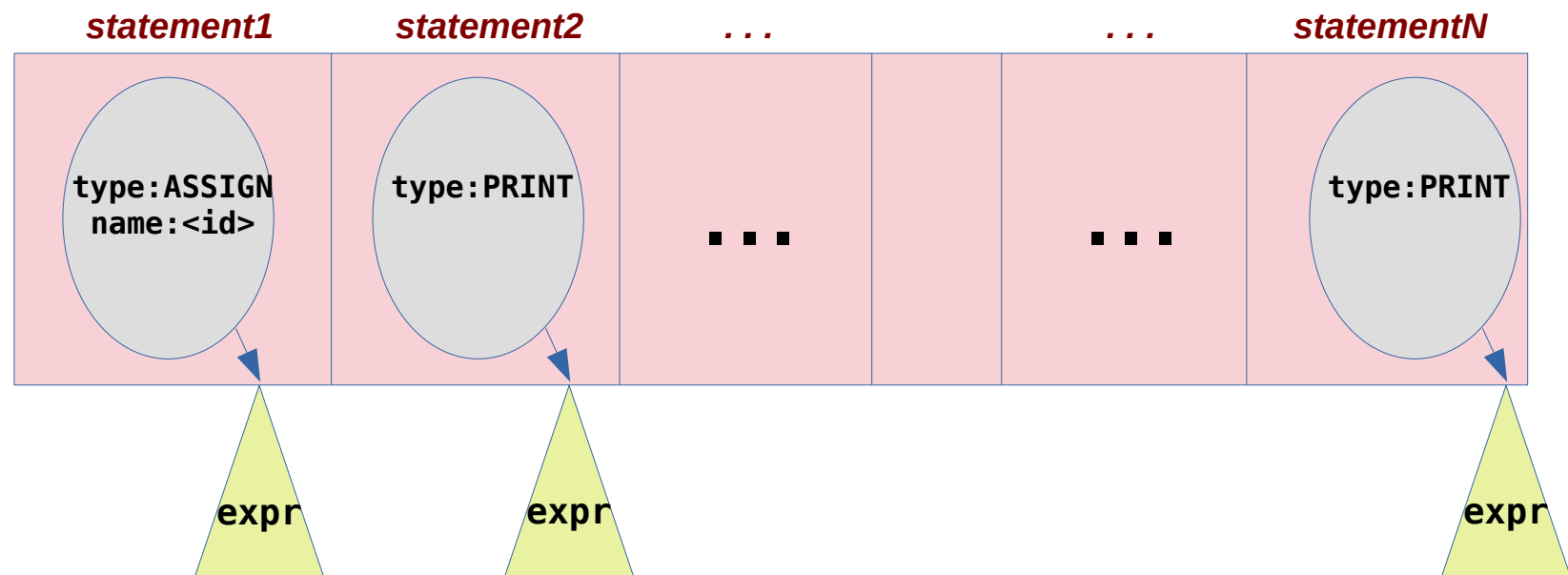
- Η μέθοδος **Stmt_list()** συνενώνει όλα τα επιμέρους AST των εντολών σε μια **λίστα**
 - Ακολουθία AST με διάταξη ίδια **με τη σειρά εμφάνισης των εντολών**
- Σε περίπτωση που ακολουθείται ο κανόνας **Stmt_list** $\rightarrow$ ϵ επιστρέφεται η κενή λίστα []
- Όταν ακολουθείται ο κανόνας **Stmt_list** $\rightarrow$ **Stmt Stmt_list** επιστρέφεται νέα λίστα
 - Με πρώτο στοιχείο το AST που επιστρέφει η **Stmt**
 - Και υπόλοιπα στοιχεία ό,τι επιστρέφει η **Stmt_list**

Στο τέλος της συντακτικής ανάλυσης

- Η μέθοδος `parse()` επιστρέφει τη λίστα (ακολουθία) των AST των εντολών που προέκυψε κατά τη συντακτική ανάλυση
- Η λίστα των AST θα χρησιμοποιηθεί σε επόμενο βήμα του μεταγλωττιστή
 - Π.χ. θα εκτελεστεί απευθείας σε έναν διερμηνευτή (interpreter)
 - Ή θα χρησιμοποιηθεί σε οποιονδήποτε άλλον μετασχηματισμό του κώδικα εισόδου

Παράδειγμα: διερμηνεία εντολών

- Το τελικό αποτέλεσμα της συντακτικής ανάλυσης είναι μια **λίστα** με τις εντολές του προγράμματος
 - Με τη σειρά εμφάνισης στον πηγαίο κώδικα
 - Κάθε στοιχείο της λίστας είναι ένα AST με ρίζα έναν κόμβο **ASSIGN** ή **PRINT**
 - Κατά τη διερμηνεία εκτελούμε ένα-ένα τα AST της λίστας



AST αριθμητικής έκφρασης (expr)

- Τα φύλλα είναι
 - NUMBER: Αριθμητικές σταθερές (τιμές float)
 - DEREf: Αναφορές σε μεταβλητές
 - Κατά τον υπολογισμό, κάθε εμφάνιση μεταβλητής πρέπει να αντικατασταθεί από την **τρέχουσα τιμή** της
- Οι ενδιάμεσοι κόμβοι είναι
 - OP: Αριθμητικοί δυαδικοί τελεστές που συνδυάζουν τις τιμές του **αριστερού** και **δεξιού** υποδένδρου
 - Ανάλογα με το είδος του τελεστή υπολογίζεται μια νέα τιμή

Τιμές μεταβλητών

- Στη γλώσσα που αναπτύσσουμε οι μεταβλητές δεν δηλώνονται
 - Υπάρχει ένας μοναδικός τύπος δεδομένων: **float**
 - Που αποθηκεύονται οι τιμές των μεταβλητών;
 - Κατά τη διάρκεια της διερμηνείας χρησιμοποιούμε ένα λεξικό (dict) ως **πίνακα συμβόλων** (symbol table) με κλειδιά τα ονόματα των μεταβλητών και τιμές τις τιμές των μεταβλητών
 - Αρχικά ο πίνακας συμβόλων είναι κενός
 - Γεμίζει με τις **εντολές ανάθεσης** (κόμβος **ASSIGN**) τιμές σε μεταβλητή
 - Όταν συναντήσουμε κόμβο **DEREF**, αναζητούμε το όνομα της μεταβλητής στο symbol table και επιστρέφουμε την τιμή που βρίσκεται μέσα
 - **Σφάλμα Εκτέλεσης** (Run Error) αν η μεταβλητή δεν βρεθεί στο symbol table (δεν έχει οριστεί ακόμα)

Υπολογισμός αριθμητικής έκφρασης

- Διάσχιση δένδρου με **DFS**
 - Προτεραιότητα κατά βάθος
- Επεξεργασία **post-order**
 - Αφού επιστρέψουμε από τα παιδιά
 - Υπολογισμός πράξης και προώθηση προς τα πίσω του αποτελέσματος

